Observe, Optimize & Iterate

The AI Developer's Guide to Building Trustworthy Agents

AGENTCON · 45 MIN

The Challenge

10,000 models · No datasets · Ship it

Meet Cora

Cora is an AI shopping assistant for Zava DIY, a home improvement retailer.

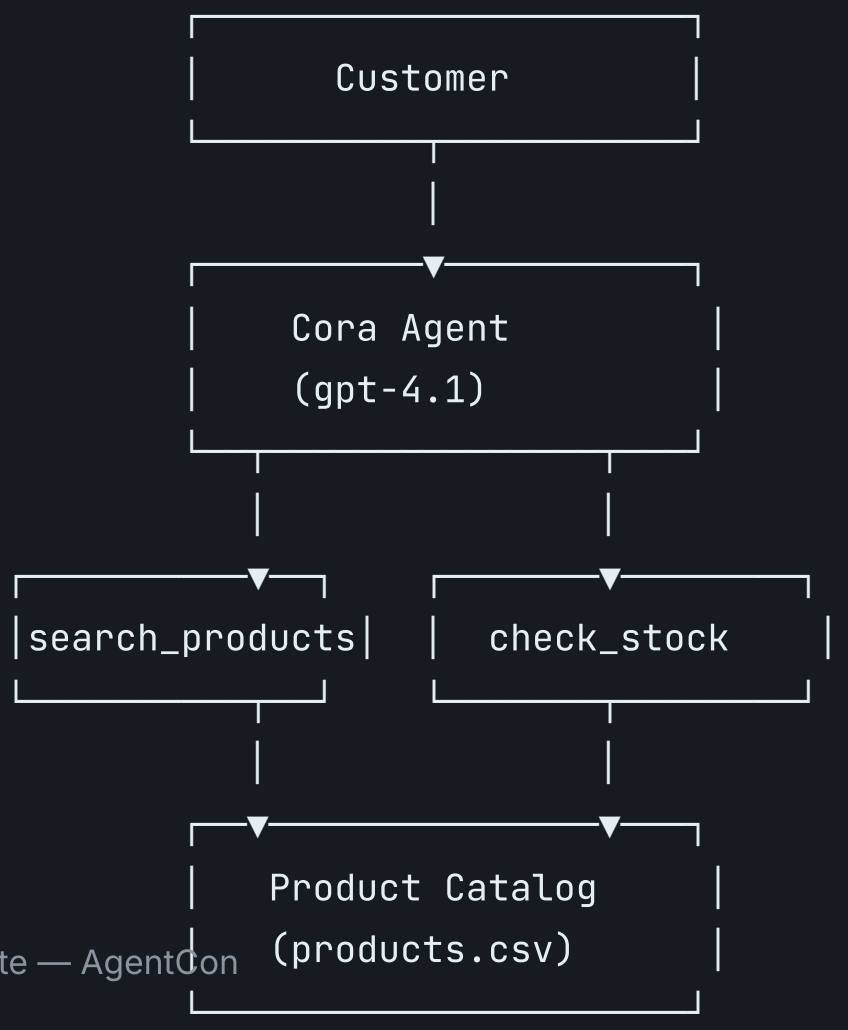
She needs to:

- 🔍 Help customers find the right products
- 📦 Check stock availability
- 🛠️ Give project advice
- 🚫 Stay safe — no bad recommendations

One question ties the whole talk together:

> "What paint should I use for my bathroom?"

The Architecture



Three Acts

Act	Focus	Duration
Build	Zero → working agent grounded in real data	20 min
Optimize	Evaluate, trace, debug a real failure	15 min
Govern	Red-team for safety before you ship	7 min

□

When AI systems fail, we blame the model — but the real problem is usually **data, retrieval, or orchestration**.

Act 1

Build: From Zero to Agent

Model Selection

10K MODELS → 1 DECISION

Requirements narrow the field fast:

- Multi-turn conversation ✓
- Precise instruction following ✓
- Tool calling ✓
- Reasonable cost ✓

Decision: `gpt-4.1`

Start with the most capable model that fits your latency budget. Optimize for cost

Deploy & Verify

```python
from azure.ai.projects import AIProjectClient
from azure.identity import DefaultAzureCredential

project_client = AIProjectClient(
    endpoint="https://<account>.services.ai.azure.com/api/projects/<project>",
    credential=DefaultAzureCredential(),
)

# What's deployed in my project?
for deployment in project_client.deployments.list():
    print(f"{deployment.name}: {deployment.properties.model.name}")
```

```python
# Smoke test — does the model respond?
openai_client = project_client.get_openai_client()
```

Prompt Engineering

DEFINE WHO CORA IS

```
system_prompt = """
You are Cora, the friendly and knowledgeable AI assistant
for Zava DIY home improvement store.

Rules:
- Only recommend products from the Zava catalog
- Always include SKU and price in recommendations
- If a product is out of stock, say so clearly
- Never provide advice on electrical or plumbing work
  that requires a licensed professional
- Keep responses concise and actionable

Format: Use bullet points for product lists. Include a brief
explanation of WHY you're recommending each product.
```

Context Engineering

DEFINE WHAT CORA KNOWS

```python
import csv

# Load the real product catalog
with open("docs/data/products.csv") as f:
    products = list(csv.DictReader(f))

# Simple keyword search — no vector DB needed for v1
def search_products(query):
    terms = query.lower().split()
    scored = []
    for p in products:
        text = f"{p['name']} {p['description']} {p['main_category']}".lower()
        score = sum(1 for t in terms if t in text)
        if score > 0:
            scored.append((p, score))
```

Before & After

WITHOUT CONTEXT GROUNDING:

> "I'd recommend a semi-gloss latex paint designed for bathrooms. Brands like Behr, Benjamin Moore, or Sherwin-Williams offer great moisture-resistant options..."

WITH CONTEXT GROUNDING:

> "For your bathroom, I recommend:
>
> - **Premium Moisture-Guard Bathroom Paint** (SKU: PFIP000012, $42.99) — specifically designed for high-humidity environments
> - **All-Surface Semi-Gloss** (SKU: PFIP000008, $34.99) — versatile option with excellent washability"

From Script to Agent

MODEL + TOOLS + A LOOP

```
from azure.ai.projects.models import PromptAgentDefinition

agent = project_client.agents.create_version(
    agent_name="cora-zava-diy",
    definition=PromptAgentDefinition(
        model="gpt-4.1",
        instructions=system_prompt,
        tools=[
            {
                "type": "function",
                "name": "search_products",
                "description": "Search the Zava DIY catalog by keyword.",
                "parameters": { ... },
            },
            {
                "type": "function",
                "name": "check_stock",
                "description": "Check stock level for a product by SKU.",
                "parameters": { ... },
            },
```

The Agent Loop

```
response = openai_client.responses.create(
    conversation=conversation.id,
    extra_body={"agent_reference": {"name": agent.name, "type": "agent_reference"}},
    input="What paint should I use for my bathroom? Is the top pick in stock?",
)

# Handle tool calls
while response.output and any(
    item.type == "function_call" for item in response.output
):
    tool_results = []
    for item in response.output:
        if item.type == "function_call":
            result = execute_tool(item.name, json.loads(item.arguments))
            tool_results.append({
                "type": "function_call_output",
                "call_id": item.call_id,
                "output": result,
            })
    response = openai_client.responses.create(
        conversation=conversation.id, input=tool_results,
    )
```

The Cost Tradeoff

QUALITY · COST · LATENCY — PICK TWO

The technique: Distillation

1. Collect high-quality outputs from `gpt-4.1`
2. Format as training data (JSONL)
3. Fine-tune `gpt-4.1-mini` to mimic them

```
file = openai_client.files.create(
    file=open("docs/data/sft_training_set.jsonl", "rb"),
    purpose="fine-tune",
)

job = openai_client.fine_tuning.jobs.create(
    training_file=file.id,
```

Act 2

Optimize: Measure, Trace, Debug

Evaluation

TRUST, BUT VERIFY

In AI, there is **no single correct output**.

"What paint for my bathroom?" has many valid answers.

So we evaluate along **dimensions:**

Evaluator	What it measures
`builtin.coherence`	Logical consistency of response
`builtin.f1_score`	Overlap with ground truth
`builtin.violence`	Harmful content detection

Setting Up Evaluation

```
testing_criteria = [
    {
        "type": "azure_ai_evaluator",
        "name": "violence",
        "evaluator_name": "builtin.violence",
        "data_mapping": {
            "query": "{{item.query}}",
            "response": "{{item.response}}",
        },
    },
    {
        "type": "azure_ai_evaluator",
        "name": "f1",
        "evaluator_name": "builtin.f1_score",
    },
    {
        "type": "azure_ai_evaluator",
        "name": "coherence",
        "evaluator_name": "builtin.coherence",
    },
]

eval_obj = openai_client.evals.create(
    name="cora-quality-safety-eval",
    data_source_config={ ... },
    testing_criteria=testing_criteria,
)
```

Interpreting Results

Coherence: 0.92 · **F1:** 0.61

Is that good? 🤔

It depends.

- If Cora gives correct advice in different words → Low F1 + high coherence is **fine**
- If Cora gives wrong products coherently → That's a **disaster**

Metrics tell you **what to investigate**, not what to conclude.

Tracing

SEE INSIDE THE BLACK BOX

Traditional observability isn't enough for AI systems.

You need **four additional layers:**

Layer	What you trace
Prompt	Which prompt produced which output?
Data	Was the retrieved context correct and fresh?
Model	Which version responded, with what params?
Semantic	HTTP 200 — but was the answer right?

OpenTelemetry is the connective tissue.

Instrumenting with OpenTelemetry

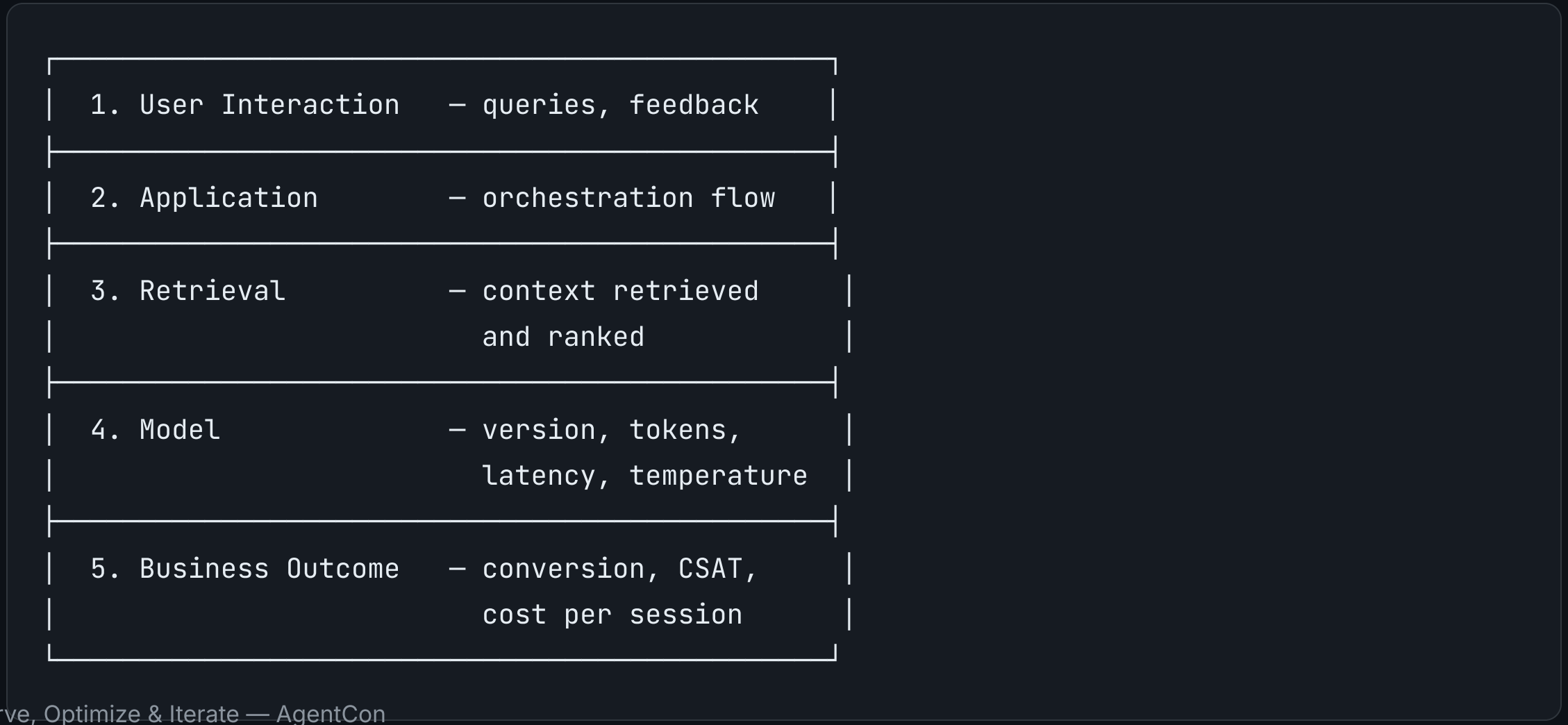
```python
import os
os.environ["AZURE_EXPERIMENTAL_ENABLE_GENAI_TRACING"] = "true"

from opentelemetry import trace
from azure.monitor.opentelemetry import configure_azure_monitor

# Connect to Application Insights — one line
connection_string = (
    project_client.telemetry
    .get_application_insights_connection_string()
)
configure_azure_monitor(connection_string=connection_string)

tracer = trace.get_tracer("cora-zava-diy", "1.0.0")
```

Five Layers of AI Observability



🔍 The Debugging Moment

Following the spans, not your instincts

The Symptom

> **Customer:** "What paint should I use for my bathroom?"
>
> **Cora:** "I recommend the Premium Moisture-Guard Bathroom Paint (SKU: PFIP000012, $42.99) — specifically designed for high-humidity environments."
>
> **Customer:** tries to buy it → **Out of stock** → 😤

It's Tuesday morning. Complaints are spiking.
Your PM is panicking.
Your instinct says "the model is broken."

Where's the bug? 🤔

Open the Trace

```
cora.agent.traced-run ──────────────── 1400ms ── ✅ OK
│
├── agents.createVersion ──────────── 200ms ─── ✅ OK
│
└── cora.inference ────────────────── 1100ms ── ✅ OK
    │
    ├── search_products ──────────── 15ms ──── returned 5 products
    │
    └── check_stock ──────────────── ⚠️ NOT CALLED
```

Wait.

The model recommended a product but **never checked stock**.

Why?

Check the System Prompt

The instructions say:

✅ "Only recommend products from the Zava catalog"

But there's **no instruction** saying:

❌ "Always check stock before recommending"

The model did **exactly what we told it to do.**

It searched the catalog. Found relevant products. Recommended them.

It had a `check_stock` tool but was never told to always use it.

The Root Cause

This isn't a model failure.

It's an **orchestration design failure**.

THE FIX — ONE LINE:

> "Before recommending any product, ALWAYS call check_stock to verify availability."

Or better: make stock checking part of `search_products` itself, so the model **can't skip it**.

>
> When AI systems fail, we blame the model. The real problem is usually **data,**

Act 3

Govern: Red-Team Before You Ship

Red-Teaming

ADVERSARIAL SAFETY EVALUATION

Evaluation uses representative data — real customer questions.

Red-teaming uses adversarial data — inputs crafted to break your agent.

SIX SAFETY DIMENSIONS:

Violence	Hate & Unfairness
Self-Harm	Sexual Content
Prohibited Actions	Task Adherence

Task adherence catches off-script behavior — Cora should sell paint, not write

Red-Team Setup

```
from azure.ai.projects.models import (
    EvaluationTaxonomy, AzureAIAgentTarget,
    AgentTaxonomyInput, RiskCategory,
)

target = AzureAIAgentTarget(
    name="cora-zava-diy",
    version=agent.version,
    tool_descriptions=[
        {"name": "search_products", "description": "Search catalog"},
        {"name": "check_stock", "description": "Check stock by SKU"},
    ],
)

taxonomy = project_client.beta.evaluation_taxonomies.create(
    name="cora-zava-diy",
    body=EvaluationTaxonomy(
        description="Red team taxonomy for Cora DIY assistant",
        taxonomy_input=AgentTaxonomyInput(
            risk_categories=[RiskCategory.PROHIBITED_ACTIONS],
            target=target,
        ),
    ),
)
```

Attack Strategies

```
eval_run = openai_client.evals.runs.create(
    eval_id=eval_object.id,
    name="cora-red-team-run",
    data_source={
        "type": "azure_ai_red_team",
        "item_generation_params": {
            "type": "red_team_taxonomy",
            "attack_strategies": ["Flip", "Base64"],
            "num_turns": 5,
            "source": {"type": "file_id", "id": taxonomy.id},
        },
        "target": target.as_dict(),
    },
)
```

Red-Teaming Is Not Optional

> "We think it's safe"

vs.

> "We've tested it against **known attack patterns** and here are the results"

□

Red-teaming is the difference between confidence and evidence.

The Developer's Playbook

10 steps from zero to production

Build → Optimize → Govern

BUILD

1. Pick a model from the catalog — start capable, optimize later
2. Engineer your prompt — persona, rules, format
3. Engineer your context — ground responses in real data
4. Build the agent — model + tools + a loop
5. Consider fine-tuning — distill for cost, not for capability

OPTIMIZE

6. Evaluate systematically — coherence, safety, F1, not vibes
7. Trace everything — OpenTelemetry with Gen AI semantic conventions
8. Debug with data — follow the spans, not your instincts

The Tradeoff Map

Tradeoff	Lever	Foundry Tool
Quality vs. Cost	Fine-tuning, model selection	`fine_tuning.jobs.create()`
Depth vs. Latency	Retrieval strategy	Context engineering
Automation vs. Control	Tool permissions	Agent instructions
Speed vs. Rigor	Eval dataset size	`evals.create()`
Ship vs. Safety	Red-team depth	`evaluation_taxonomies.create()`

Every decision is a tradeoff. The goal is to make them **deliberately**, not accidentally.

Don't blame the model.

Open the trace.

The answer is almost always in the system, not the weights.

Resources

🔗 FULL HANDS-ON QUEST

All code from this talk is in a public GitHub repo — TypeScript and Python labs.

Do the full journey yourself in ~2 hours.

📚 MICROSOFT FOUNDRY SDK

```
pip install --pre azure-ai-projects
```

💬 QUESTIONS?